

Análise do pacote Open Source Requests

Introdução

Eu escolhi analisar a biblioteca Requests, que é uma biblioteca Open Source escrita em Python e utilizada para fazer requisições HTTP. Eu escolhi o Requests porque já tinha utilizado essa biblioteca em uma atividade anterior do estágio, quando precisei acessar uma API e pegar informações que estavam sendo retornadas por ela.

O Requests facilita a comunicação entre um programa e um servidor. Com ele, podemos fazer requisições para endereços da internet, receber informações, enviar dados, trabalhar com respostas em JSON, utilizar cabeçalhos, autenticação, cookies e também definir um tempo máximo para uma requisição.

Instalação

Antes de utilizar o Requests, é necessário instalá-lo. Isso pode ser feito pelo terminal utilizando o pip.

O comando para instalar é ***python -m pip install requests***.

Também é possível instalar utilizando ***pip install requests***, que foi a forma que utilizei durante meu projeto.

Depois da instalação, a biblioteca pode ser importada normalmente no Python utilizando `import requests`.

Uso geral da biblioteca

O uso mais básico do Requests é fazer uma requisição para um endereço, podemos utilizar o método **`get()`**.

Por exemplo, podemos ter **`resposta = requests.get("https://exemplo.com")`**.

Nesse caso, o programa faz uma requisição GET para o endereço informado e guarda o resultado na variável **`resposta`**.

A partir dessa resposta, podemos acessar várias informações. O Requests cria um objeto Response, que possui informações sobre o que foi recebido do servidor.

Uma dessas informações é o código de status. Podemos acessar isso utilizando **`resposta.status_code`**.

Por exemplo, quando uma requisição é realizada com sucesso, normalmente recebemos o código 200. Outros códigos podem indicar redirecionamentos ou erros.

Também podemos acessar o conteúdo recebido usando **resposta.text**. Quando o conteúdo retornado está em JSON, podemos utilizar **resposta.json()** para transformar esse conteúdo em dados que podem ser utilizados no Python.

O Requests não trabalha apenas com GET. A biblioteca também possui métodos como **post()**, **put()**, **delete()**, **head()** e **options()**. Isso permite que ela seja utilizada em diferentes situações envolvendo comunicação com servidores.

Como utilizei o Requests

No projeto que fiz anteriormente, utilizei o Requests para acessar uma API que retornava informações de usuários.

A ideia principal era fazer a conexão com a API e depois pegar os dados que ela retornava.

No código, utilizei **import requests** para importar a biblioteca. Depois defini o endereço da API em uma variável e fiz a requisição utilizando **requests.get(url)**.

A resposta foi armazenada em uma variável e depois utilizei **resposta.json()** para conseguir trabalhar com os dados recebidos.

A API retornava várias informações dentro de um JSON. Eu então acessei a parte que continha os usuários e percorri os dados para mostrar cada pessoa separadamente.

Isso foi importante para mim durante a análise porque eu já tinha visto o funcionamento do Requests na prática. No começo eu sabia apenas que **requests.get()** fazia a conexão com a API, mas não entendia muito bem o que existia por trás dessa chamada.

A resposta da requisição

Quando fazemos uma requisição utilizando o Requests, o resultado não é simplesmente uma string ou um dicionário. A biblioteca retorna um objeto **Response**.

Esse objeto reúne várias informações da resposta recebida.

O **status_code** permite verificar o código HTTP retornado. O **text** permite acessar o conteúdo como texto e o **content** permite acessar o conteúdo em bytes.

Quando trabalhamos com APIs que retornam JSON, podemos utilizar **json()** para interpretar esse conteúdo.

Também existe o **headers**, que permite acessar os cabeçalhos enviados pelo servidor. Outra informação disponível é a URL final da resposta, além de informações relacionadas a redirecionamentos.

Uma coisa que achei importante entender é que receber um JSON não significa necessariamente que a requisição deu certo. Uma API pode retornar um JSON contendo uma mensagem de erro, por exemplo. Por isso, também podemos verificar o **status_code** ou utilizar **raise_for_status()** para verificar se ocorreu um erro HTTP.

Timeout e tratamento de erros

Outra parte que analisei foi o **timeout**.

Quando fazemos uma requisição, pode acontecer de o servidor demorar muito para responder. Para evitar que o programa fique esperando indefinidamente, podemos definir um tempo para a requisição.

Por exemplo, podemos utilizar **requests.get(url, timeout=10)**.

Nesse caso, existe um limite relacionado ao tempo de espera pela resposta. Se ocorrer um problema desse tipo, o Requests pode gerar uma exceção **Timeout**.

A biblioteca também possui outras exceções, como **ConnectionError**, **HTTPError**, **TooManyRedirects** e outras que fazem parte de **RequestException**.

No meu projeto, utilizei **try** e **except** para tratar esses problemas. Assim, se a API não respondesse corretamente, o programa poderia mostrar uma mensagem de erro em vez de simplesmente parar.

Essa foi uma parte que consegui testar na prática. Também testei uma situação em que a conexão demorava para responder para verificar se o tratamento de **Timeout** realmente funcionava.

Outras funções do Requests

Além do que utilizei diretamente, encontrei outras funções durante a pesquisa.

O Requests permite enviar informações em uma requisição utilizando **POST**, por exemplo. Também é possível enviar dados em JSON, definir headers, utilizar cookies, autenticação e enviar arquivos.

Uma outra parte interessante é o **Session**.

Uma sessão pode ser utilizada quando o programa precisa realizar várias requisições e queremos manter algumas informações entre elas, como cookies e determinadas configurações. A própria documentação apresenta o **requests.Session()** como uma forma de manter esse tipo de informação e também aproveitar o connection pooling.

Eu não precisei utilizar Session no meu projeto, mas incluí na análise porque faz parte da estrutura da biblioteca e mostra que o Requests possui recursos além do simples **get()**.

Estrutura do projeto

Depois de entender a utilização da biblioteca, fui analisar como o projeto é organizado por dentro.

O código-fonte do Requests está disponível publicamente no GitHub. Atualmente, o repositório possui uma pasta **src/requests**, que contém o código principal da biblioteca. Também existem pastas como **tests**, utilizada para os testes, e **docs**, que contém a documentação do projeto.

Além dessas pastas, o projeto possui arquivos como **README.md**, **HISTORY.md**, **LICENSE**, **setup.py** e **pyproject.toml**.

O **README.md** apresenta o projeto e mostra informações básicas sobre sua utilização. O **HISTORY.md** registra mudanças e versões anteriores. O **LICENSE** apresenta a licença do projeto.

O **pyproject.toml** é especialmente importante porque contém informações de configuração do pacote, como o nome do projeto, a versão do Python suportada e suas dependências.

O que existe dentro de src/requests

A pasta **src/requests** é uma das partes mais importantes para entender o funcionamento da biblioteca, porque é nela que está o código utilizado quando fazemos **import requests**.

Dentro dela existem diferentes arquivos responsáveis por partes específicas do funcionamento.

Entre eles estão arquivos relacionados às requisições, respostas, sessões, autenticação, cookies, exceções e outras funções utilizadas pela biblioteca.

Essa separação é importante porque evita colocar todo o funcionamento da biblioteca em um único arquivo. Cada parte pode cuidar de uma responsabilidade diferente e depois essas partes trabalham juntas.

Um exemplo disso é o objeto **Response**, que é utilizado para representar a resposta recebida depois de uma requisição. Outro exemplo é o **Session**, que possui funções relacionadas ao uso de sessões.

Dependências

O Requests também não funciona completamente sozinho. O projeto possui algumas dependências que são instaladas junto com a biblioteca.

No **pyproject.toml**, aparecem dependências como **urllib3**, **certifi**, **charset_normalizer** e **idna**.

O **urllib3**, por exemplo, está relacionado à comunicação HTTP. O **certifi** fornece certificados utilizados para verificar conexões HTTPS. Já **charset_normalizer** e **idna** participam de outras partes do tratamento de dados e URLs.

Isso mostra uma coisa interessante sobre bibliotecas Open Source: mesmo quando instalamos apenas um pacote, ele pode depender de outras bibliotecas para realizar determinadas tarefas.

Analisando o funcionamento interno

Para entender melhor o funcionamento interno, é importante observar que quando escrevemos algo simples como **requests.get(url)**, várias coisas acontecem dentro da biblioteca.

O método **get()** faz parte da interface que o Requests oferece para o usuário. Ele recebe a URL e outros parâmetros possíveis e retorna um objeto **Response**. A documentação da API mostra que **requests.get()** retorna justamente um objeto desse tipo.

Depois disso, o objeto **Response** pode ser utilizado para acessar o código de status, conteúdo, headers, cookies, URL e outras informações.

Então, mesmo que para o usuário o código pareça simples, existe uma estrutura maior por trás dele. O Requests precisa lidar com a criação da requisição, comunicação com o servidor, recebimento da resposta, tratamento dos dados e possíveis erros.

É justamente isso que torna uma biblioteca útil: em vez de o programador precisar implementar toda essa comunicação HTTP manualmente, ele utiliza funções prontas que escondem essa complexidade.

Clean Room Design

Na parte de Clean Room Design, a ideia é pensar em como seria possível criar uma biblioteca com comportamento parecido com o Requests sem copiar o código original.

Para isso, primeiro é necessário observar o que a biblioteca faz quando é utilizada.

Por exemplo, sabemos que podemos chamar **get()** passando uma URL e que o resultado é um objeto que possui informações como **status_code**, **text** e **json()**.

A partir dessas informações, seria possível criar uma biblioteca própria com uma interface semelhante.

Ela poderia possuir uma função **get(url)** para realizar uma requisição e retornar um objeto **Response**.

Esse objeto poderia possuir um **status_code** para guardar o código retornado pelo servidor, um **text** para acessar o conteúdo em texto e uma função **json()** para transformar uma resposta JSON em dados utilizáveis pelo Python.

Também seria possível adicionar um parâmetro **timeout** para definir quanto tempo o programa deve esperar por uma resposta e criar exceções próprias para situações como timeout ou falha na conexão.

O objetivo não seria copiar a implementação do Requests. A ideia seria observar o comportamento que o usuário consegue utilizar e criar uma implementação independente que tenha uma função parecida.

Isso também mostra que, para criar uma biblioteca desse tipo, não basta saber apenas como chamar uma função. É necessário entender quais informações entram, quais informações saem e como o programa deve se comportar quando alguma coisa dá errado.

O que consegui entender com a análise

Antes de fazer essa pesquisa, eu conhecia o Requests principalmente pela utilização de **requests.get()** para acessar uma API. Depois da análise, consegui entender melhor que essa função é apenas uma parte de uma biblioteca bem maior.

Também consegui entender melhor o objeto Response, o funcionamento de **status_code**, **text** e **json()**, além do tratamento de erros e do **timeout**.

Outra coisa que chamou minha atenção foi a organização do projeto. No começo, quando usamos uma biblioteca, vemos apenas o **import** e algumas funções. Quando entramos no repositório, percebemos que existe uma separação entre código, testes, documentação e configurações.

No meu caso, a análise ficou mais fácil porque consegui relacionar o que encontrei no código-fonte com o projeto que eu já tinha feito usando uma API.

Conclusão

A análise do Requests ajudou a entender melhor como uma biblioteca Open Source pode ser construída e organizada.

Além de aprender sobre o funcionamento das requisições HTTP, consegui relacionar a pesquisa com algo que já tinha feito na prática. Isso facilitou principalmente a compreensão de **get()**, **Response**, **json()**, **status_code** e **timeout**.

Também foi possível perceber que uma função aparentemente simples pode depender de várias outras partes para funcionar. O Requests possui seu próprio código, testes, documentação e dependências, todos organizados dentro do projeto.

Na parte de Clean Room Design, consegui entender que é possível estudar o comportamento de uma biblioteca e pensar em uma implementação própria sem precisar copiar seu código e que é necessário entender o que cada função recebe, o que retorna e como deve se comportar em diferentes situações.